FLET FRAMEWORK: COMPLETE REFERENCE GUIDE (Part 2 - Controls, Properties & Advanced Topics)

Table of Contents (Part 2)

1. [Complete Page Reference](#1-complete-page-reference)
2. [Layout Controls](#2-layout-controls)
3. [Input Controls](#3-input-controls)
4. [Display Controls](#4-display-controls)
5. [Button Controls](#5-button-controls)
6. [Navigation Controls](#6-navigation-controls)
7. [Animation Controls](#7-animation-controls)
8. [Theme & Styling](#8-theme--styling)
9. [Window Management](#9-window-management)
10. [State Management Deep Dive](#10-state-management-deep-dive)
11. [Database Integration](#11-database-integration)
12. [Testing Flet Apps](#12-testing-flet-apps)
13. [Environment Variables](#13-environment-variables)
14. [Packaging All Platforms](#14-packaging-all-platforms)
15. [Icons Reference](#15-icons-reference)
16. [Colors Reference](#16-colors-reference)

---

1. Complete Page Reference

Page Properties

```python
import flet as ft

def main(page: ft.Page):
    # Core properties
    page.title = "My App"
    page.route = "/home"  # Current route
    page.views = []  # Navigation stack
    page.controls = []  # Controls on the page

    # Theme properties
    page.theme_mode = ft.ThemeMode.LIGHT  # LIGHT, DARK, SYSTEM
    page.theme = ft.Theme()  # Custom theme
    page.dark_theme = ft.Theme()  # Dark theme
    page.bgcolor = ft.Colors.WHITE

    # Layout properties
    page.padding = 20
    page.spacing = 10
    page.horizontal_alignment = ft.CrossAxisAlignment.CENTER
    page.vertical_alignment = ft.MainAxisAlignment.CENTER
    page.scroll = ft.ScrollMode.AUTO

    # Window properties (desktop)
    page.window.width = 800
    page.window.height = 600
    page.window.min_width = 400
    page.window.min_height = 300
    page.window.max_width = 1200
    page.window.max_height = 900
    page.window.resizable = True
    page.window.full_screen = False
    page.window.focused = True
    page.window.title = "My App"
    page.window.icon = "icon.ico"
    page.window.always_on_top = False
    page.window.skip_task_bar = False
    page.window.prevent_close = False
    page.window.opacity = 1.0
    page.window.frameless = False
    page.window.draggable = True
    page.window.toolbar_visible = True
    page.window.title_bar_buttons_visible = True
```

```
    # Web properties
    page.web = False  # True if running in browser
    page.web_window_name = "_blank"

    # Platform detection
    page.platform = ft.PagePlatform.ANDROID  # IOS, MACOS, WINDOWS, LINUX, WEB

    # Accessibility
    page.show_semantics_debugger = False

    # Auto-update
    page.auto_update = True
    page.update_interval = 0.1  # Seconds between updates

    # Session & Auth
    page.session = {}  # Server-side session storage
    page.auth = None  # Authentication data
    page.client_storage = None  # Client-side storage

    # PubSub
    page.pubsub = None  # Publish-Subscribe system

    # Adaptive
    page.adaptive = False

    # Debug
    page.debug = False

ft.run(main)
```

Page Methods

```python
# Control management
page.add(control1, control2, ...)  # Add controls
page.remove(control)  # Remove control
page.clean()  # Remove all controls
page.update()  # Send updates to client

# Navigation
page.navigate(route, **params)  # Navigate to route
await page.push_route(route, **params)  # Push route to stack
page.go(route)  # Navigate (alternative)
page.get_route()  # Get current route

# Views
page.views.clear()  # Clear view stack
page.views.append(view)  # Add view
page.views.pop()  # Remove top view

# Pop multiple views
await page.pop_views_until(route, result=None)

# Dialogs
page.show_dialog(dialog)  # Show dialog
page.open(dialog)  # Open dialog (alias)
page.close_dialog()  # Close dialog
page.pop_dialog()  # Pop dialog

# URLs
page.launch_url(url)  # Open URL in browser

# Background tasks
page.run_task(coroutine, *args)  # Run async task
page.run_thread(function, *args)  # Run in thread

# Authentication
page.login(provider, **kwargs)  # Login with OAuth
page.logout()  # Logout

# Keyboard
page.on_keyboard_event = handler  # Keyboard events
```

```
# Resize
page.on_resize = handler  # Window resize events

# Close
page.on_close = handler  # App close events

# Error
page.on_error = handler  # Error events
```

Page Events

```python
def main(page: ft.Page):
    # Route change event
    def on_route_change(e: ft.RouteChangeEvent):
        print(f"Route: {e.route}")
    page.on_route_change = on_route_change

    # View pop event
    def on_view_pop(e: ft.ViewPopEvent):
        print(f"View: {e.view}")
    page.on_view_pop = on_view_pop

    # Views pop until event
    def on_views_pop_until(e: ft.ViewsPopUntilEvent):
        print(f"Result: {e.result}")
    page.on_views_pop_until = on_views_pop_until

    # Keyboard event
    def on_keyboard(e: ft.KeyboardEvent):
        print(f"Key: {e.key}")
    page.on_keyboard_event = on_keyboard

    # Resize event
    def on_resize(e: ft.WindowResizeEvent):
        print(f"Size: {e.width}x{e.height}")
    page.on_resize = on_resize

    # Close event
    def on_close(e: ft.WindowCloseEvent):
        print("App closing")
    page.on_close = on_close

    # Error event
    def on_error(e: ft.ErrorEvent):
        print(f"Error: {e.data}")
    page.on_error = on_error

    # Login event
    def on_login(e: ft.LoginEvent):
        if e.error:
            print(f"Login error: {e.error}")
        else:
            print(f"User: {page.auth.user}")
    page.on_login = on_login

    # Logout event
    def on_logout(e: ft.LogoutEvent):
        print("Logged out")
    page.on_logout = on_logout

ft.run(main)
```

---

2. Layout Controls

Row

```python
```

```python
import flet as ft

row = ft.Row(
    controls=[
        ft.Text("Item 1"),
        ft.Text("Item 2"),
        ft.Text("Item 3"),
    ],
    alignment=ft.MainAxisAlignment.CENTER,  # START, END, CENTER, SPACE_BETWEEN, SPACE_AROUND, SPACE_EVENLY
    vertical_alignment=ft.CrossAxisAlignment.CENTER,  # START, END, CENTER, STRETCH, BASELINE
    spacing=10,  # Space between children
    wrap=False,  # Wrap to next line if overflow
    scroll=ft.ScrollMode.AUTO,  # AUTO, ALWAYS, HIDDEN
    expand=True,  # Expand to fill available space
    width=400,
    height=200,
    visible=True,
    disabled=False,
    animate_opacity=300,
    animate_scale=ft.animation.Animation(300, ft.AnimationCurve.EASE),
    animate_rotation=ft.animation.Animation(300, ft.AnimationCurve.EASE),
    animate_offset=ft.animation.Animation(300, ft.AnimationCurve.EASE),
    animate_position=ft.animation.Animation(300, ft.AnimationCurve.EASE),
)

# Dynamic adding
row.controls.append(ft.Text("New Item"))
row.update()

# Remove
row.controls.remove(row.controls[0])
row.update()
```

Column

```python
column = ft.Column(
    controls=[
        ft.Text("Item 1"),
        ft.Text("Item 2"),
        ft.Text("Item 3"),
    ],
    alignment=ft.MainAxisAlignment.CENTER,
    horizontal_alignment=ft.CrossAxisAlignment.CENTER,
    spacing=10,
    scroll=ft.ScrollMode.AUTO,
    expand=True,
)

# Adding children
for i in range(10):
    column.controls.append(ft.Text(f"Item {i}"))
column.update()
```

Stack

```python
stack = ft.Stack(
    controls=[
        ft.Container(
            width=200,
            height=200,
            bgcolor=ft.Colors.RED,
            left=0,
            top=0,
        ),
        ft.Container(
            width=100,
            height=100,
            bgcolor=ft.Colors.BLUE,
            left=50,
```

```
            top=50,
        ),
        ft.Container(
            width=50,
            height=50,
            bgcolor=ft.Colors.GREEN,
            left=75,
            top=75,
        ),
    ],
    width=400,
    height=400,
    alignment=ft.alignment.center,
)

# Positioning in Stack
ft.Container(
    content=ft.Text("Positioned"),
    left=100,  # Position from left
    top=100,   # Position from top
    right=None,  # Position from right
    bottom=None,  # Position from bottom
    width=100,
    height=50,
)
```

Container

```python
container = ft.Container(
    content=ft.Text("Hello"),
    width=200,
    height=100,
    padding=20,
    margin=10,
    border=ft.Border.all(2, ft.Colors.BLACK),
    border_radius=ft.BorderRadius.all(10),
    bgcolor=ft.Colors.WHITE,
    shadow=ft.BoxShadow(
        spread_radius=1,
        blur_radius=5,
        color=ft.Colors.GREY,
        offset=ft.Offset(2, 2),
    ),
    alignment=ft.alignment.center,
    on_click=lambda e: print("Clicked"),
    on_hover=lambda e: print(f"Hover: {e.data}"),
    animate=ft.animation.Animation(300, ft.AnimationCurve.EASE),
    animate_opacity=300,
    animate_scale=ft.animation.Animation(300, ft.AnimationCurve.EASE),
    animate_rotation=ft.animation.Animation(300, ft.AnimationCurve.EASE),
)

# Border styles
border_sides = ft.Border(
    left=ft.BorderSide(color=ft.Colors.RED, width=2),
    right=ft.BorderSide(color=ft.Colors.BLUE, width=2),
    top=ft.BorderSide(color=ft.Colors.GREEN, width=2),
    bottom=ft.BorderSide(color=ft.Colors.ORANGE, width=2),
)

# Border radius
border_radius = ft.BorderRadius(
    top_left=10,
    top_right=20,
    bottom_left=30,
    bottom_right=40,
)
```

ResponsiveRow

```python
responsive_row = ft.ResponsiveRow(
    controls=[
        ft.Container(
            content=ft.Text("Column 1"),
            col={"xs": 12, "sm": 6, "md": 4, "lg": 3, "xl": 2},
        ),
        ft.Container(
            content=ft.Text("Column 2"),
            col={"xs": 12, "sm": 6, "md": 4, "lg": 3, "xl": 2},
        ),
        ft.Container(
            content=ft.Text("Column 3"),
            col={"xs": 12, "sm": 12, "md": 4, "lg": 6, "xl": 8},
        ),
    ],
    spacing=10,
    run_spacing=10,
    vertical_alignment=ft.CrossAxisAlignment.START,
)

# Column sizes
# xs: extra small (<600px) - 12 columns
# sm: small (600-960px) - 6 columns
# md: medium (960-1280px) - 4 columns
# lg: large (1280-1920px) - 3 columns
# xl: extra large (>1920px) - 2 columns
```

GridView

```python
grid_view = ft.GridView(
    controls=[
        ft.Container(
            content=ft.Text(f"Item {i}"),
            alignment=ft.alignment.center,
            bgcolor=ft.Colors.AMBER_100,
            border=ft.Border.all(1, ft.Colors.AMBER_400),
            border_radius=ft.BorderRadius.all(5),
        )
        for i in range(20)
    ],
    expand=True,
    max_extent=150,  # Maximum width of each tile
    child_aspect_ratio=1.0,  # Width/height ratio
    runs_count=0,  # Number of rows/columns (0 = auto)
    spacing=5,
    run_spacing=5,
    padding=10,
    scroll=ft.ScrollMode.AUTO,
)
```

ListView

```python
list_view = ft.ListView(
    controls=[
        ft.ListTile(
            title=ft.Text(f"Item {i}"),
            subtitle=ft.Text(f"Subtitle {i}"),
            leading=ft.Icon(ft.Icons.FOLDER),
            trailing=ft.Icon(ft.Icons.ARROW_FORWARD),
        )
        for i in range(50)
    ],
    expand=True,
    spacing=10,
    item_extent=50,  # Fixed height for performance
    first_item_prototype=False,  # Use first item as size reference
    padding=10,
    divider_thickness=1,
```

```
    auto_scroll=False,
    scroll=ft.ScrollMode.AUTO,
)
```

Tabs

```python
tabs = ft.Tabs(
    selected_index=0,
    tabs=[
        ft.Tab(
            text="Tab 1",
            content=ft.Container(
                content=ft.Text("Content 1"),
                padding=20,
            ),
        ),
        ft.Tab(
            text="Tab 2",
            content=ft.Container(
                content=ft.Text("Content 2"),
                padding=20,
            ),
        ),
        ft.Tab(
            text="Tab 3",
            content=ft.Container(
                content=ft.Text("Content 3"),
                padding=20,
            ),
        ),
    ],
    expand=True,
    animation_duration=300,
    divider_color=ft.Colors.GREY,
    label_color=ft.Colors.BLUE,
    unselected_label_color=ft.Colors.GREY,
    indicator_color=ft.Colors.BLUE,
    indicator_padding=ft.Padding(10, 0, 10, 0),
)
```

DataTable

```python
data_table = ft.DataTable(
    columns=[
        ft.DataColumn(
            ft.Text("Name"),
            numeric=False,
            tooltip="Name column",
            on_sort=lambda e: print("Sorting"),
        ),
        ft.DataColumn(
            ft.Text("Age"),
            numeric=True,
        ),
        ft.DataColumn(
            ft.Text("Country"),
        ),
    ],
    rows=[
        ft.DataRow(
            cells=[
                ft.DataCell(ft.Text("John")),
                ft.DataCell(ft.Text("25")),
                ft.DataCell(ft.Text("USA")),
            ],
            on_select_changed=lambda e: print(f"Selected: {e.data}"),
        ),
        ft.DataRow(
            cells=[
```

```
                ft.DataCell(ft.Text("Jane")),
                ft.DataCell(ft.Text("30")),
                ft.DataCell(ft.Text("UK")),
            ],
        ),
    ],
    heading_row_color=ft.Colors.GREY_100,
    horizontal_margin=10,
    column_spacing=20,
    divider_thickness=1,
    sort_column_index=0,
    sort_ascending=True,
)

# DataCell properties
ft.DataCell(
    content=ft.Text("Value"),
    show_edit_icon=True,
    placeholder=True,
)
```

Card

```python
card = ft.Card(
    content=ft.Container(
        content=ft.Column([
            ft.ListTile(
                leading=ft.Icon(ft.Icons.ALBUM),
                title=ft.Text("Card Title"),
                subtitle=ft.Text("Card subtitle"),
            ),
            ft.Row([
                ft.TextButton("Action 1"),
                ft.TextButton("Action 2"),
            ], alignment=ft.MainAxisAlignment.END),
        ]),
        width=300,
        padding=10,
    ),
    elevation=5,
    color=ft.Colors.WHITE,
    shadow_color=ft.Colors.GREY,
    surface_tint_color=ft.Colors.BLUE,
    shape=ft.RoundedRectangleBorder(radius=10),
    clip_behavior=ft.ClipBehavior.ANTI_ALIAS,
    margin=10,
    on_click=lambda e: print("Card clicked"),
)
```

ExpansionTile

```python
expansion_tile = ft.ExpansionTile(
    title=ft.Text("Expand me"),
    subtitle=ft.Text("Click to expand"),
    leading=ft.Icon(ft.Icons.FOLDER),
    trailing=ft.Icon(ft.Icons.ARROW_DROP_DOWN),
    controls=[
        ft.ListTile(title=ft.Text("Item 1")),
        ft.ListTile(title=ft.Text("Item 2")),
        ft.ListTile(title=ft.Text("Item 3")),
    ],
    expanded=True,
    initially_expanded=True,
    maintain_state=True,
    on_expansion_change=lambda e: print(f"Expanded: {e.data}"),
)

# ExpansionPanelList
panel_list = ft.ExpansionPanelList(
```

```
    panels=[
        ft.ExpansionPanel(
            header=ft.ListTile(title=ft.Text("Panel 1")),
            body=ft.Text("Content 1"),
            is_expanded=True,
        ),
        ft.ExpansionPanel(
            header=ft.ListTile(title=ft.Text("Panel 2")),
            body=ft.Text("Content 2"),
            is_expanded=False,
        ),
    ],
    elevation=2,
    divider_color=ft.Colors.GREY,
    animation_duration=300,
)
```

NavigationRail

```python
navigation_rail = ft.NavigationRail(
    selected_index=0,
    label_type=ft.NavigationRailLabelType.ALL,
    extended=False,
    leading=ft.Icon(ft.Icons.HOME),
    trailing=ft.Icon(ft.Icons.SETTINGS),
    destinations=[
        ft.NavigationRailDestination(
            icon=ft.Icons.HOME,
            selected_icon=ft.Icons.HOME_FILLED,
            label="Home",
        ),
        ft.NavigationRailDestination(
            icon=ft.Icons.STORE,
            selected_icon=ft.Icons.STORE_FILLED,
            label="Store",
        ),
        ft.NavigationRailDestination(
            icon=ft.Icons.PERSON,
            selected_icon=ft.Icons.PERSON_FILLED,
            label="Profile",
        ),
    ],
    on_change=lambda e: print(f"Selected: {e.control.selected_index}"),
    min_width=56,
    min_extended_width=200,
    group_alignment=-1,
)
```

NavigationDrawer

```python
drawer = ft.NavigationDrawer(
    selected_index=0,
    elevation=16,
    controls=[
        ft.Container(height=12),
        ft.NavigationDrawerDestination(
            label="Home",
            icon=ft.Icons.HOME,
            selected_icon=ft.Icons.HOME_FILLED,
        ),
        ft.Divider(thickness=2),
        ft.NavigationDrawerDestination(
            label="Settings",
            icon=ft.Icons.SETTINGS,
            selected_icon=ft.Icons.SETTINGS,
        ),
        ft.NavigationDrawerDestination(
            label="About",
            icon=ft.Icons.INFO,
```

```
            selected_icon=ft.Icons.INFO,
        ),
    ],
    on_change=lambda e: print(f"Selected: {e.control.selected_index}"),
    indicator_color=ft.Colors.BLUE,
    bgcolor=ft.Colors.WHITE,
)

# Show drawer
await page.show_drawer()
```

SafeArea

```python
safe_area = ft.SafeArea(
    content=ft.Text("Safe content"),
    left=True,
    top=True,
    right=True,
    bottom=True,
    minimum=ft.Padding(20, 20, 20, 20),
)
```

Padding, Align, Center, SizedBox

```python
# Padding
padded = ft.Padding(
    content=ft.Text("Padded content"),
    padding=20,
)

# Align
aligned = ft.Align(
    content=ft.Text("Aligned content"),
    alignment=ft.alignment.top_left,
)

# Center
centered = ft.Center(
    content=ft.Text("Centered content"),
    width=200,
    height=100,
)

# SizedBox
sized_box = ft.SizedBox(
    content=ft.Text("Fixed size"),
    width=100,
    height=50,
)

# ConstrainedBox
constrained = ft.ConstrainedBox(
    content=ft.Text("Constrained"),
    min_width=50,
    max_width=200,
    min_height=50,
    max_height=100,
)
```

---

3. Input Controls

TextField

```python
text_field = ft.TextField(
    value="Initial value",
```

```
    label="Enter text",
    hint_text="Type something...",
    helper_text="Helper text",
    error_text="Error message",
    prefix_text="Prefix",
    suffix_text="Suffix",
    prefix_icon=ft.Icons.PERSON,
    suffix_icon=ft.Icons.CLEAR,
    icon=ft.Icons.EMAIL,
    icon_color=ft.Colors.BLUE,
    label_style=ft.TextStyle(size=16),
    hint_style=ft.TextStyle(color=ft.Colors.GREY),
    helper_style=ft.TextStyle(size=12),
    error_style=ft.TextStyle(color=ft.Colors.RED),
    text_style=ft.TextStyle(size=18),
    text_align=ft.TextAlign.LEFT,  # LEFT, RIGHT, CENTER, JUSTIFY, START, END
    keyboard_type=ft.KeyboardType.TEXT,  # TEXT, NUMBER, PHONE, EMAIL, URL, MULTILINE
    text_capitalization=ft.TextCapitalization.NONE,  # NONE, CHARACTERS, WORDS, SENTENCES
    multiline=False,
    min_lines=1,
    max_lines=5,
    max_length=100,
    counter_text="0/100",
    filled=True,
    fill_color=ft.Colors.GREY_100,
    border=ft.InputBorder.OUTLINE,  # NONE, UNDERLINE, OUTLINE
    border_color=ft.Colors.BLUE,
    focused_border_color=ft.Colors.DARK_BLUE,
    border_width=1,
    focused_border_width=2,
    hover_color=ft.Colors.GREY_50,
    bgcolor=ft.Colors.WHITE,
    selection_color=ft.Colors.BLUE,
    cursor_color=ft.Colors.BLUE,
    cursor_width=2,
    cursor_height=20,
    autofocus=True,
    read_only=False,
    disabled=False,
    show_password=False,
    password=False,
    can_reveal_password=True,
    on_change=lambda e: print(e.control.value),
    on_submit=lambda e: print("Submitted"),
    on_focus=lambda e: print("Focused"),
    on_blur=lambda e: print("Blurred"),
    on_icon_click=lambda e: print("Icon clicked"),
)

# TextField with password visibility toggle
password_field = ft.TextField(
    password=True,
    can_reveal_password=True,
    label="Password",
)

# Number input
number_field = ft.TextField(
    keyboard_type=ft.KeyboardType.NUMBER,
    label="Age",
)

# Multiline
multiline_field = ft.TextField(
    multiline=True,
    min_lines=3,
    max_lines=10,
    label="Description",
)
```

Dropdown

```python
dropdown = ft.Dropdown(
    value="option1",
    label="Select option",
    hint_text="Choose...",
    helper_text="Select one option",
    options=[
        ft.dropdown.Option("option1", "Option 1"),
        ft.dropdown.Option("option2", "Option 2"),
        ft.dropdown.Option("option3", "Option 3"),
        ft.dropdown.Option(key="disabled", text="Disabled", disabled=True),
    ],
    width=200,
    icon=ft.Icons.ARROW_DROP_DOWN,
    icon_size=24,
    icon_enabled_color=ft.Colors.BLUE,
    icon_disabled_color=ft.Colors.GREY,
    focused_color=ft.Colors.BLUE,
    focused_bgcolor=ft.Colors.GREY_100,
    focused_border_color=ft.Colors.BLUE,
    focused_border_width=2,
    bgcolor=ft.Colors.WHITE,
    border=ft.InputBorder.OUTLINE,
    border_color=ft.Colors.GREY,
    border_width=1,
    elevation=4,
    alignment=ft.alignment.center,
    text_style=ft.TextStyle(size=16),
    label_style=ft.TextStyle(size=14),
    hint_style=ft.TextStyle(color=ft.Colors.GREY),
    helper_style=ft.TextStyle(size=12),
    error_style=ft.TextStyle(color=ft.Colors.RED),
    error_text=None,
    disabled=False,
    autofocus=False,
    on_change=lambda e: print(f"Selected: {e.control.value}"),
    on_focus=lambda e: print("Focused"),
    on_blur=lambda e: print("Blurred"),
)
```

Checkbox

```python
checkbox = ft.Checkbox(
    value=True,
    label="Accept terms",
    label_style=ft.TextStyle(size=16),
    check_color=ft.Colors.WHITE,
    active_color=ft.Colors.BLUE,
    inactive_color=ft.Colors.GREY,
    hover_color=ft.Colors.BLUE_50,
    focus_color=ft.Colors.BLUE_100,
    splash_radius=20,
    autofocus=False,
    disabled=False,
    on_change=lambda e: print(f"Checked: {e.control.value}"),
    on_focus=lambda e: print("Focused"),
    on_blur=lambda e: print("Blurred"),
)
```

Radio

```python
radio_group = ft.RadioGroup(
    value="option1",
    content=ft.Column([
        ft.Radio(value="option1", label="Option 1"),
        ft.Radio(value="option2", label="Option 2"),
        ft.Radio(value="option3", label="Option 3"),
    ]),
    on_change=lambda e: print(f"Selected: {e.data}"),
```

```
    disabled=False,
)

# Individual Radio
radio = ft.Radio(
    value="value",
    label="Label",
    label_position=ft.LabelPosition.RIGHT,  # LEFT, RIGHT
    active_color=ft.Colors.BLUE,
    inactive_color=ft.Colors.GREY,
    hover_color=ft.Colors.BLUE_50,
    focus_color=ft.Colors.BLUE_100,
    splash_radius=20,
    autofocus=False,
    disabled=False,
)
```

Switch

```python
switch = ft.Switch(
    value=True,
    label="Enable feature",
    label_style=ft.TextStyle(size=16),
    active_color=ft.Colors.BLUE,
    active_track_color=ft.Colors.BLUE_100,
    inactive_color=ft.Colors.GREY,
    inactive_track_color=ft.Colors.GREY_100,
    hover_color=ft.Colors.BLUE_50,
    focus_color=ft.Colors.BLUE_100,
    splash_radius=20,
    autofocus=False,
    disabled=False,
    on_change=lambda e: print(f"Switched: {e.control.value}"),
    on_focus=lambda e: print("Focused"),
    on_blur=lambda e: print("Blurred"),
)
```

Slider

```python
slider = ft.Slider(
    value=50,
    min=0,
    max=100,
    divisions=10,
    label="{value}%",
    thumb_color=ft.Colors.BLUE,
    active_color=ft.Colors.BLUE,
    inactive_color=ft.Colors.GREY_300,
    thumb_shape=ft.CircleBorder(),
    overlay_color=ft.Colors.BLUE_50,
    hover_color=ft.Colors.BLUE_100,
    focus_color=ft.Colors.BLUE_200,
    autofocus=False,
    disabled=False,
    on_change=lambda e: print(f"Value: {e.control.value}"),
    on_change_start=lambda e: print("Started"),
    on_change_end=lambda e: print("Ended"),
)
```

RangeSlider

```python
range_slider = ft.RangeSlider(
    min=0,
    max=100,
    values=[20, 80],
    divisions=10,
    label="{value}",
```

```
    thumb_color=ft.Colors.BLUE,
    active_color=ft.Colors.BLUE,
    inactive_color=ft.Colors.GREY_300,
    overlay_color=ft.Colors.BLUE_50,
    hover_color=ft.Colors.BLUE_100,
    focus_color=ft.Colors.BLUE_200,
    disabled=False,
    on_change=lambda e: print(f"Values: {e.control.values}"),
    on_change_start=lambda e: print("Started"),
    on_change_end=lambda e: print("Ended"),
)
```

ColorPicker

```python
color_picker = ft.ColorPicker(
    value=ft.Colors.BLUE,
    width=200,
    height=200,
    on_change=lambda e: print(f"Color: {e.control.value}"),
    on_submit=lambda e: print("Submitted"),
    disabled=False,
)

# ColorPicker with picker
color_picker_with_picker = ft.ColorPicker(
    value=ft.Colors.RED,
    picker_type=ft.ColorPickerType.PRIMARY,  # PRIMARY, SECONDARY, TERTIARY
    display_picker=True,
    show_controls=True,
    show_color_name=True,
)
```

DatePicker / TimePicker

```python
import datetime

# DatePicker
date_picker = ft.DatePicker(
    value=datetime.datetime.now(),
    first_date=datetime.datetime(2020, 1, 1),
    last_date=datetime.datetime(2030, 12, 31),
    on_change=lambda e: print(f"Date: {e.control.value}"),
    on_dismiss=lambda e: print("Dismissed"),
    on_submit=lambda e: print("Submitted"),
    help_text="Select a date",
    cancel_text="Cancel",
    confirm_text="OK",
    field_hint_text="MM/DD/YYYY",
    field_label_text="Date",
    entry_mode=ft.DatePickerEntryMode.OVERLAY,  # OVERLAY, DIALOG, INPUT
)

# TimePicker
time_picker = ft.TimePicker(
    value=datetime.datetime.now(),
    on_change=lambda e: print(f"Time: {e.control.value}"),
    on_dismiss=lambda e: print("Dismissed"),
    on_submit=lambda e: print("Submitted"),
    help_text="Select time",
    cancel_text="Cancel",
    confirm_text="OK",
    entry_mode=ft.TimePickerEntryMode.OVERLAY,  # OVERLAY, DIALOG, INPUT
    hour_label_text="Hour",
    minute_label_text="Minute",
)

# Show pickers
page.show_dialog(date_picker)
page.show_dialog(time_picker)
```

```

### FilePicker

```python
file_picker = ft.FilePicker(
    on_result=lambda e: print(f"Files: {e.files}"),
    on_upload=lambda e: print(f"Upload: {e.data}"),
    on_cancel=lambda e: print("Cancelled"),
)

# Pick files
await file_picker.pick_files(
    allow_multiple=True,
    allowed_extensions=["txt", "pdf", "jpg"],
    dialog_title="Select files",
    initial_directory="/home/user",
)

# Pick single file
await file_picker.pick_files(
    allow_multiple=False,
    allowed_extensions=["txt"],
)

# Pick directory
await file_picker.get_directory_path(
    dialog_title="Select directory",
    initial_directory="/home/user",
)

# Save file
await file_picker.save_file(
    file_name="myfile.txt",
    allowed_extensions=["txt"],
    dialog_title="Save file",
    initial_directory="/home/user",
)

# Upload files
file_picker.upload(
    files=[
        ft.FilePickerUploadFile(
            name="file1.txt",
            upload_url="http://localhost:8000/upload",
            method="PUT",
            on_progress=lambda e: print(f"Progress: {e.progress}"),
        )
    ]
)

# FilePickerResult properties
result = file_picker.result
if result:
    for file in result.files:
        print(f"Name: {file.name}")
        print(f"Size: {file.size}")
        print(f"Path: {file.path}")
        print(f"Extension: {file.extension}")
        print(f"Bytes: {file.read()}")
```

Autocomplete

```python
autocomplete = ft.Autocomplete(
    options=[
        ft.AutocompleteOption(key="apple", value="Apple"),
        ft.AutocompleteOption(key="banana", value="Banana"),
        ft.AutocompleteOption(key="cherry", value="Cherry"),
    ],
    on_select=lambda e: print(f"Selected: {e.control.selected}"),
    text_field=ft.TextField(label="Search fruits"),
```
```

```
    width=300,
    max_height=200,
    elevation=4,
    bgcolor=ft.Colors.WHITE,
    content_padding=ft.Padding(10, 5, 10, 5),
    option_style=ft.TextStyle(size=16),
    disabled=False,
)
```

ChipsInput

```python
chips_input = ft.ChipsInput(
    label="Add tags",
    hint_text="Type and press enter",
    value=["python", "flet", "fastapi"],
    max_chips=10,
    chip_label_style=ft.TextStyle(size=14),
    chip_bgcolor=ft.Colors.BLUE_50,
    chip_delete_icon=ft.Icons.CLOSE,
    chip_color=ft.Colors.BLUE,
    chip_padding=ft.Padding(8, 4, 8, 4),
    chip_border_radius=ft.BorderRadius.all(4),
    on_change=lambda e: print(f"Chips: {e.control.value}"),
    on_add=lambda e: print(f"Added: {e.data}"),
    on_delete=lambda e: print(f"Deleted: {e.data}"),
    on_blur=lambda e: print("Blurred"),
    disabled=False,
)
```

---

4. Display Controls

Text

```python
text = ft.Text(
    value="Hello World",
    size=24,
    color=ft.Colors.BLUE,
    weight=ft.FontWeight.BOLD,  # THIN, EXTRA_LIGHT, LIGHT, NORMAL, MEDIUM, SEMI_BOLD, BOLD, EXTRA_BOLD, BLACK
    italic=True,
    font_family="Arial",
    text_align=ft.TextAlign.CENTER,  # LEFT, RIGHT, CENTER, JUSTIFY, START, END
    max_lines=2,
    overflow=ft.TextOverflow.ELLIPSIS,  # CLIP, ELLIPSIS, FADE, VISIBLE
    selectable=True,
    semantics_label="Hello World",
    spans=[
        ft.TextSpan(
            text="Styled ",
            style=ft.TextStyle(size=20, color=ft.Colors.RED),
        ),
        ft.TextSpan(
            text="text",
            style=ft.TextStyle(size=20, color=ft.Colors.BLUE),
            on_click=lambda e: print("Clicked"),
            on_hover=lambda e: print(f"Hover: {e.data}"),
        ),
    ],
)

# TextStyle
style = ft.TextStyle(
    size=20,
    color=ft.Colors.BLUE,
    weight=ft.FontWeight.BOLD,
    italic=True,
    font_family="Arial",
    decoration=ft.TextDecoration.UNDERLINE,  # NONE, UNDERLINE, OVERLINE, LINE_THROUGH
```

```
    decoration_color=ft.Colors.RED,
    decoration_style=ft.TextDecorationStyle.SOLID,  # SOLID, DOUBLE, DOTTED, DASHED, WAVY
    letter_spacing=1.5,
    word_spacing=2.0,
    height=1.5,
    leading_distribution=ft.TextLeadingDistribution.EVEN,
)
```

Image

```python
image = ft.Image(
    src="https://example.com/image.jpg",
    src_base64="...base64...",  # Alternative to src
    width=200,
    height=200,
    fit=ft.ImageFit.CONTAIN,  # CONTAIN, COVER, FILL, FIT_HEIGHT, FIT_WIDTH, NONE, SCALE_DOWN
    repeat=ft.ImageRepeat.NO_REPEAT,  # NO_REPEAT, REPEAT, REPEAT_X, REPEAT_Y
    border_radius=ft.BorderRadius.all(10),
    gapless_playback=True,
    filter_quality=ft.FilterQuality.HIGH,  # LOW, MEDIUM, HIGH, NONE
    semantics_label="Image description",
    on_click=lambda e: print("Image clicked"),
    on_hover=lambda e: print(f"Hover: {e.data}"),
)

# Local image
local_image = ft.Image(
    src="/assets/image.jpg",
    width=200,
    height=200,
)
```

Icon

```python
icon = ft.Icon(
    name=ft.Icons.HOME,
    size=48,
    color=ft.Colors.BLUE,
    semantics_label="Home icon",
    on_click=lambda e: print("Icon clicked"),
    on_hover=lambda e: print(f"Hover: {e.data}"),
)

# Cupertino icon
cupertino_icon = ft.Icon(
    name=ft.CupertinoIcons.HOME,
    size=48,
    color=ft.Colors.BLUE,
)
```

CircleAvatar

```python
circle_avatar = ft.CircleAvatar(
    content=ft.Text("JD"),
    foreground_image_url="https://example.com/avatar.jpg",
    background_image_url="https://example.com/bg.jpg",
    radius=50,
    min_radius=20,
    max_radius=100,
    bgcolor=ft.Colors.BLUE,
    foreground_image_fit=ft.ImageFit.COVER,
    on_click=lambda e: print("Avatar clicked"),
    on_hover=lambda e: print(f"Hover: {e.data}"),
)
```

Chip

```python
chip = ft.Chip(
    label=ft.Text("Chip label"),
    leading=ft.Icon(ft.Icons.ACCOUNT_CIRCLE),
    avatar=ft.CircleAvatar(
        content=ft.Text("A"),
        bgcolor=ft.Colors.BLUE,
    ),
    trailing=ft.Icon(ft.Icons.CLOSE),
    selected=True,
    selectable=True,
    color=ft.Colors.BLUE,
    bgcolor=ft.Colors.WHITE,
    selected_color=ft.Colors.WHITE,
    selected_bgcolor=ft.Colors.BLUE,
    shape=ft.RoundedRectangleBorder(radius=10),
    elevation=2,
    padding=ft.Padding(10, 5, 10, 5),
    label_padding=ft.Padding(5, 0, 5, 0),
    on_select=lambda e: print(f"Selected: {e.control.selected}"),
    on_click=lambda e: print("Chip clicked"),
)
```

Badge

```python
badge = ft.Badge(
    content=ft.Icon(ft.Icons.NOTIFICATIONS),
    text="99+",
    bgcolor=ft.Colors.RED,
    text_color=ft.Colors.WHITE,
    offset=ft.Offset(10, -10),
    padding=ft.Padding(4, 2, 4, 2),
    border=ft.Border.all(1, ft.Colors.WHITE),
    border_radius=ft.BorderRadius.all(10),
    visible=True,
    on_click=lambda e: print("Badge clicked"),
)
```

Tooltip

```python
tooltip = ft.Tooltip(
    content=ft.Icon(ft.Icons.HELP),
    message="This is a tooltip",
    bgcolor=ft.Colors.BLACK,
    text_color=ft.Colors.WHITE,
    padding=ft.Padding(10, 5, 10, 5),
    border_radius=ft.BorderRadius.all(5),
    margin=ft.Padding(5, 5, 5, 5),
    wait_duration=500,  # milliseconds
    show_duration=2000,
    vertical_offset=10,
    prefer_below=True,
)
```

ProgressBar

```python
progress_bar = ft.ProgressBar(
    value=0.5,  # 0.0 to 1.0
    bar_height=10,
    color=ft.Colors.BLUE,
    bgcolor=ft.Colors.GREY_200,
    border_radius=ft.BorderRadius.all(5),
    animation_duration=300,
    tooltip="50%",
    indeterminate=False,
)
```

```
# Indeterminate (loading)
indeterminate_progress = ft.ProgressBar(
    indeterminate=True,
    bar_height=10,
    color=ft.Colors.BLUE,
)
```

ProgressRing

```python
progress_ring = ft.ProgressRing(
    value=0.75,  # 0.0 to 1.0
    width=50,
    height=50,
    stroke_width=4,
    color=ft.Colors.BLUE,
    bgcolor=ft.Colors.GREY_200,
    rotation=0.0,
    animation_duration=300,
    tooltip="75%",
    indeterminate=False,
)

# Indeterminate
indeterminate_ring = ft.ProgressRing(
    indeterminate=True,
    width=50,
    height=50,
    stroke_width=4,
    color=ft.Colors.BLUE,
)
```

Divider

```python
divider = ft.Divider(
    height=20,
    thickness=2,
    color=ft.Colors.GREY,
    indent=20,
    end_indent=20,
    leading=ft.Text("or"),
)

# VerticalDivider
vertical_divider = ft.VerticalDivider(
    width=20,
    thickness=2,
    color=ft.Colors.GREY,
    indent=20,
    end_indent=20,
)
```

SnackBar

```python
snack_bar = ft.SnackBar(
    content=ft.Text("Message"),
    bgcolor=ft.Colors.BLACK,
    action=ft.TextButton(
        "OK",
        on_click=lambda e: page.close_dialog(),
    ),
    action_color=ft.Colors.WHITE,
    close_icon_color=ft.Colors.WHITE,
    behavior=ft.SnackBarBehavior.FLOATING,  # FIXED, FLOATING
    width=400,
    elevation=6,
    duration=3000,
```

```
    on_visible=lambda e: print("Snackbar visible"),
)

# Show snackbar
page.show_snack_bar(snack_bar)
```

Banner

```python
banner = ft.Banner(
    content=ft.Text("This is a banner message"),
    bgcolor=ft.Colors.BLUE,
    color=ft.Colors.WHITE,
    leading=ft.Icon(ft.Icons.INFO),
    actions=[
        ft.TextButton("Action 1", on_click=lambda e: print("Action 1")),
        ft.TextButton("Action 2", on_click=lambda e: print("Action 2")),
    ],
    padding=ft.Padding(20, 10, 20, 10),
    force_actions_below=True,
)

# Show banner
page.show_banner(banner)

# Hide banner
page.hide_banner()
```

AlertDialog

```python
alert_dialog = ft.AlertDialog(
    title=ft.Text("Dialog Title"),
    content=ft.Text("Dialog content"),
    actions=[
        ft.TextButton("Cancel", on_click=lambda e: page.close_dialog()),
        ft.TextButton("OK", on_click=lambda e: page.close_dialog()),
    ],
    actions_alignment=ft.MainAxisAlignment.END,
    title_padding=ft.Padding(20, 20, 20, 0),
    content_padding=ft.Padding(20, 10, 20, 10),
    modal=True,  # Blocks outside interactions
    shape=ft.RoundedRectangleBorder(radius=10),
    bgcolor=ft.Colors.WHITE,
    elevation=24,
)

# Show dialog
page.show_dialog(alert_dialog)
# or
page.open(alert_dialog)
```

BottomSheet

```python
bottom_sheet = ft.BottomSheet(
    content=ft.Container(
        content=ft.Column([
            ft.ListTile(title=ft.Text("Option 1")),
            ft.ListTile(title=ft.Text("Option 2")),
            ft.ListTile(title=ft.Text("Option 3")),
        ]),
        padding=20,
    ),
    bgcolor=ft.Colors.WHITE,
    elevation=8,
    shape=ft.RoundedRectangleBorder(
        radius=ft.BorderRadius.only(top_left=10, top_right=10)
    ),
    show_drag_handle=True,
```

```
    drag_handle_color=ft.Colors.GREY,
    enable_drag=True,
    on_dismiss=lambda e: print("Sheet dismissed"),
)

# Show bottom sheet
page.show_bottom_sheet(bottom_sheet)
```

---

5. Button Controls

ElevatedButton

```python
elevated_button = ft.ElevatedButton(
    text="Click me",
    icon=ft.Icons.ADD,
    icon_color=ft.Colors.WHITE,
    color=ft.Colors.WHITE,
    bgcolor=ft.Colors.BLUE,
    elevation=4,
    highlight_elevation=8,
    disabled_elevation=0,
    shadow_color=ft.Colors.GREY,
    style=ft.ButtonStyle(
        shape=ft.RoundedRectangleBorder(radius=10),
        padding=ft.Padding(20, 10, 20, 10),
        side=ft.BorderSide(color=ft.Colors.BLUE, width=2),
        text_style=ft.TextStyle(size=16),
        overlay_color=ft.Colors.BLUE_50,
        animation_duration=300,
        elevation=ft.ButtonStyleProperty(4),
        bgcolor=ft.ButtonStyleProperty(ft.Colors.BLUE),
        color=ft.ButtonStyleProperty(ft.Colors.WHITE),
    ),
    on_click=lambda e: print("Button clicked"),
    on_hover=lambda e: print(f"Hover: {e.data}"),
    on_long_press=lambda e: print("Long press"),
    autofocus=False,
    disabled=False,
    expand=True,
    width=200,
    height=50,
)
```

FilledButton

```python
filled_button = ft.FilledButton(
    text="Filled Button",
    icon=ft.Icons.ADD,
    style=ft.ButtonStyle(
        shape=ft.RoundedRectangleBorder(radius=10),
        padding=ft.Padding(20, 10, 20, 10),
    ),
    on_click=lambda e: print("Clicked"),
)
```

FilledTonalButton

```python
filled_tonal_button = ft.FilledTonalButton(
    text="Tonal Button",
    icon=ft.Icons.ADD,
    on_click=lambda e: print("Clicked"),
)
```

OutlinedButton

```python
outlined_button = ft.OutlinedButton(
    text="Outlined Button",
    icon=ft.Icons.ADD,
    style=ft.ButtonStyle(
        shape=ft.RoundedRectangleBorder(radius=10),
        side=ft.BorderSide(color=ft.Colors.BLUE, width=2),
        padding=ft.Padding(20, 10, 20, 10),
    ),
    on_click=lambda e: print("Clicked"),
)
```

TextButton

```python
text_button = ft.TextButton(
    text="Text Button",
    icon=ft.Icons.ADD,
    style=ft.ButtonStyle(
        padding=ft.Padding(20, 10, 20, 10),
        text_style=ft.TextStyle(size=16, color=ft.Colors.BLUE),
    ),
    on_click=lambda e: print("Clicked"),
)
```

IconButton

```python
icon_button = ft.IconButton(
    icon=ft.Icons.ADD,
    icon_size=24,
    icon_color=ft.Colors.BLUE,
    selected_icon=ft.Icons.REMOVE,
    selected_icon_size=24,
    selected_icon_color=ft.Colors.RED,
    tooltip="Add item",
    style=ft.ButtonStyle(
        shape=ft.CircleBorder(),
        padding=ft.Padding(10, 10, 10, 10),
        overlay_color=ft.Colors.BLUE_50,
        animation_duration=300,
    ),
    autofocus=False,
    disabled=False,
    selected=False,
    on_click=lambda e: print("Icon clicked"),
    on_hover=lambda e: print(f"Hover: {e.data}"),
)
```

PopupMenuButton

```python
popup_menu = ft.PopupMenuButton(
    icon=ft.Icons.MORE_VERT,
    icon_size=24,
    icon_color=ft.Colors.BLUE,
    items=[
        ft.PopupMenuItem(
            content=ft.Text("Item 1"),
            icon=ft.Icons.EDIT,
            on_click=lambda e: print("Item 1 clicked"),
        ),
        ft.PopupMenuItem(
            content=ft.Text("Item 2"),
            icon=ft.Icons.DELETE,
            on_click=lambda e: print("Item 2 clicked"),
        ),
        ft.PopupMenuItem(
            content=ft.Text("Item 3"),
```

```
            icon=ft.Icons.SHARE,
            on_click=lambda e: print("Item 3 clicked"),
        ),
    ],
    tooltip="Menu",
    elevation=8,
    shape=ft.RoundedRectangleBorder(radius=5),
    bgcolor=ft.Colors.WHITE,
    shadow_color=ft.Colors.GREY,
    on_select=lambda e: print(f"Selected: {e.data}"),
)
```

FloatingActionButton

```python
fab = ft.FloatingActionButton(
    icon=ft.Icons.ADD,
    text="Add",
    foreground_color=ft.Colors.WHITE,
    bgcolor=ft.Colors.BLUE,
    icon_size=24,
    mini=False,
    shape=ft.CircleBorder(),
    elevation=6,
    highlight_elevation=12,
    disabled_elevation=0,
    tooltip="Add item",
    autofocus=False,
    disabled=False,
    on_click=lambda e: print("FAB clicked"),
)

# Page FAB
page.floating_action_button = fab
page.floating_action_button_location = ft.FloatingActionButtonLocation.CENTER_FLOAT
```

---

6. Navigation Controls

AppBar

```python
app_bar = ft.AppBar(
    title=ft.Text("App Title"),
    leading=ft.IconButton(ft.Icons.MENU, on_click=lambda e: print("Menu")),
    leading_width=40,
    actions=[
        ft.IconButton(ft.Icons.SEARCH, on_click=lambda e: print("Search")),
        ft.IconButton(ft.Icons.NOTIFICATIONS, on_click=lambda e: print("Notifications")),
    ],
    bgcolor=ft.Colors.BLUE,
    color=ft.Colors.WHITE,
    center_title=True,
    elevation=4,
    shadow_color=ft.Colors.GREY,
    shape=ft.RoundedRectangleBorder(
        radius=ft.BorderRadius.only(bottom_left=10, bottom_right=10)
    ),
    automatically_imply_leading=True,
    title_spacing=20,
    toolbar_height=56,
    toolbar_text_style=ft.TextStyle(size=20, color=ft.Colors.WHITE),
    system_overlay_style=ft.SystemOverlayStyle.LIGHT,
    adaptive=True,
)

# Set page appbar
page.appbar = app_bar
```

BottomAppBar

```python
bottom_app_bar = ft.BottomAppBar(
    content=ft.Row([
        ft.IconButton(ft.Icons.HOME, on_click=lambda e: print("Home")),
        ft.IconButton(ft.Icons.SEARCH, on_click=lambda e: print("Search")),
        ft.IconButton(ft.Icons.PERSON, on_click=lambda e: print("Profile")),
    ]),
    bgcolor=ft.Colors.BLUE,
    color=ft.Colors.WHITE,
    elevation=4,
    shape=ft.RoundedRectangleBorder(
        radius=ft.BorderRadius.only(top_left=10, top_right=10)
    ),
)

# Set page bottom appbar
page.bottom_appbar = bottom_app_bar
```

NavigationBar

```python
navigation_bar = ft.NavigationBar(
    selected_index=0,
    destinations=[
        ft.NavigationBarDestination(
            icon=ft.Icons.HOME,
            selected_icon=ft.Icons.HOME_FILLED,
            label="Home",
            tooltip="Home",
        ),
        ft.NavigationBarDestination(
            icon=ft.Icons.STORE,
            selected_icon=ft.Icons.STORE_FILLED,
            label="Store",
        ),
        ft.NavigationBarDestination(
            icon=ft.Icons.PERSON,
            selected_icon=ft.Icons.PERSON_FILLED,
            label="Profile",
        ),
    ],
    on_change=lambda e: print(f"Selected: {e.control.selected_index}"),
    label_behavior=ft.NavigationBarLabelBehavior.ALWAYS_SHOW,
    elevation=4,
    bgcolor=ft.Colors.WHITE,
    indicator_color=ft.Colors.BLUE,
    indicator_shape=ft.RoundedRectangleBorder(radius=10),
    border=ft.Border(top=ft.BorderSide(color=ft.Colors.GREY, width=1)),
    animation_duration=300,
    height=56,
    shadow_color=ft.Colors.GREY,
    width=400,
)
```

Drawer

```python
drawer = ft.Drawer(
    content=ft.Column([
        ft.Container(
            content=ft.Column([
                ft.Text("Menu", size=24),
                ft.Divider(),
            ]),
            padding=20,
        ),
        ft.ListTile(
            leading=ft.Icon(ft.Icons.HOME),
            title=ft.Text("Home"),
```

```
            on_click=lambda e: print("Home"),
        ),
        ft.ListTile(
            leading=ft.Icon(ft.Icons.SETTINGS),
            title=ft.Text("Settings"),
            on_click=lambda e: print("Settings"),
        ),
        ft.ListTile(
            leading=ft.Icon(ft.Icons.INFO),
            title=ft.Text("About"),
            on_click=lambda e: print("About"),
        ),
    ]),
    elevation=16,
    width=300,
    bgcolor=ft.Colors.WHITE,
    shape=ft.RoundedRectangleBorder(
        radius=ft.BorderRadius.only(top_right=10, bottom_right=10)
    ),
)

# Set page drawer
page.drawer = drawer
```

---

7. Animation Controls

AnimatedContainer

```python
animated_container = ft.AnimatedContainer(
    content=ft.Text("Animated"),
    width=100,
    height=100,
    bgcolor=ft.Colors.BLUE,
    border_radius=ft.BorderRadius.all(10),
    duration=500,
    curve=ft.AnimationCurve.EASE_IN_OUT,  # EASE, EASE_IN, EASE_OUT, EASE_IN_OUT, LINEAR, BOUNCE, DECELERATE, ELASTIC
    animate=ft.animation.Animation(500, ft.AnimationCurve.EASE),
    on_animation_end=lambda e: print("Animation ended"),
)

# Animate properties
def animate_container(e):
    animated_container.width = 200
    animated_container.height = 200
    animated_container.bgcolor = ft.Colors.RED
    animated_container.update()
```

AnimatedOpacity

```python
animated_opacity = ft.AnimatedOpacity(
    content=ft.Text("Fade me"),
    opacity=1.0,
    duration=500,
    curve=ft.AnimationCurve.EASE,
    on_animation_end=lambda e: print("Animation ended"),
)

# Toggle opacity
def toggle_opacity(e):
    animated_opacity.opacity = 0.0 if animated_opacity.opacity == 1.0 else 1.0
    animated_opacity.update()
```

AnimatedPositioned

```python
animated_positioned = ft.AnimatedPositioned(
```

```
    content=ft.Container(
        width=100,
        height=100,
        bgcolor=ft.Colors.BLUE,
    ),
    left=0,
    top=0,
    duration=500,
    curve=ft.AnimationCurve.EASE,
)

# Move
def move_positioned(e):
    animated_positioned.left = 100
    animated_positioned.top = 100
    animated_positioned.update()
```

AnimatedSwitcher

```python
animated_switcher = ft.AnimatedSwitcher(
    content=ft.Container(
        content=ft.Text("Content 1"),
        width=200,
        height=100,
        bgcolor=ft.Colors.BLUE,
        alignment=ft.alignment.center,
    ),
    duration=500,
    switch_in_curve=ft.AnimationCurve.EASE_IN,
    switch_out_curve=ft.AnimationCurve.EASE_OUT,
    transition=ft.AnimatedSwitcherTransition.FADE,  # FADE, SCALE, ROTATE, SLIDE
    reverse=True,
)

# Switch content
def switch_content(e):
    current = animated_switcher.content
    if isinstance(current, ft.Container):
        animated_switcher.content = ft.Container(
            content=ft.Text("Content 2"),
            width=200,
            height=100,
            bgcolor=ft.Colors.RED,
            alignment=ft.alignment.center,
        )
    else:
        animated_switcher.content = ft.Container(
            content=ft.Text("Content 1"),
            width=200,
            height=100,
            bgcolor=ft.Colors.BLUE,
            alignment=ft.alignment.center,
        )
    animated_switcher.update()
```

FadeTransition

```python
fade_transition = ft.FadeTransition(
    content=ft.Text("Fade text"),
    opacity=0.5,
    animation=ft.Animation(500, ft.AnimationCurve.EASE),
)
```

ScaleTransition

```python
scale_transition = ft.ScaleTransition(
    content=ft.Text("Scale text"),
```

```
    scale=1.0,
    animation=ft.Animation(500, ft.AnimationCurve.EASE),
)
```

SlideTransition

```python
slide_transition = ft.SlideTransition(
    content=ft.Text("Slide text"),
    offset=ft.Offset(0.0, 0.0),
    animation=ft.Animation(500, ft.AnimationCurve.EASE),
)
```

RotationTransition

```python
rotation_transition = ft.RotationTransition(
    content=ft.Text("Rotate text"),
    turns=0.0,
    animation=ft.Animation(500, ft.AnimationCurve.EASE),
)
```

AnimatedBuilder

```python
class AnimatedWidget(ft.UserControl):
    def __init__(self):
        super().__init__()
        self.animation_value = 0.0
        self.animation = ft.Animation(
            duration=1000,
            curve=ft.AnimationCurve.EASE,
        )

    def build(self):
        return ft.AnimatedBuilder(
            animation=self.animation,
            builder=self._build_animated,
        )

    def _build_animated(self, _):
        return ft.Container(
            width=100 + self.animation_value * 100,
            height=100 + self.animation_value * 100,
            bgcolor=ft.Colors.BLUE,
            border_radius=ft.BorderRadius.all(10 * self.animation_value),
        )
```

---

8. Theme & Styling

Theme Configuration

```python
theme = ft.Theme(
    color_scheme_seed=ft.Colors.BLUE,
    color_scheme=ft.ColorScheme(
        primary=ft.Colors.BLUE,
        secondary=ft.Colors.GREEN,
        surface=ft.Colors.WHITE,
        background=ft.Colors.GREY_50,
        error=ft.Colors.RED,
        on_primary=ft.Colors.WHITE,
        on_secondary=ft.Colors.WHITE,
        on_surface=ft.Colors.BLACK,
        on_background=ft.Colors.BLACK,
        on_error=ft.Colors.WHITE,
    ),
```

```
font_family="Roboto",
text_theme=ft.TextTheme(
    headline_large=ft.TextStyle(size=32, weight=ft.FontWeight.BOLD),
    headline_medium=ft.TextStyle(size=28, weight=ft.FontWeight.BOLD),
    headline_small=ft.TextStyle(size=24, weight=ft.FontWeight.BOLD),
    body_large=ft.TextStyle(size=16),
    body_medium=ft.TextStyle(size=14),
    body_small=ft.TextStyle(size=12),
    label_large=ft.TextStyle(size=14, weight=ft.FontWeight.MEDIUM),
    label_medium=ft.TextStyle(size=12, weight=ft.FontWeight.MEDIUM),
    label_small=ft.TextStyle(size=10, weight=ft.FontWeight.MEDIUM),
),
button_theme=ft.ButtonTheme(
    min_width=88,
    height=36,
    padding=ft.Padding(16, 0, 16, 0),
    shape=ft.RoundedRectangleBorder(radius=4),
    aligned_dropdown=False,
    button_color=ft.Colors.BLUE,
    disabled_color=ft.Colors.GREY,
    highlight_color=ft.Colors.BLUE_50,
    splash_color=ft.Colors.BLUE_100,
    focus_color=ft.Colors.BLUE_200,
    hover_color=ft.Colors.BLUE_50,
),
card_theme=ft.CardTheme(
    elevation=2,
    margin=ft.Padding(10, 10, 10, 10),
    shape=ft.RoundedRectangleBorder(radius=8),
    clip_behavior=ft.ClipBehavior.ANTI_ALIAS,
    color=ft.Colors.WHITE,
    shadow_color=ft.Colors.GREY,
),
appbar_theme=ft.AppBarTheme(
    color=ft.Colors.BLUE,
    foreground_color=ft.Colors.WHITE,
    elevation=4,
    shadow_color=ft.Colors.GREY,
    shape=ft.RoundedRectangleBorder(),
    center_title=False,
    title_text_style=ft.TextStyle(size=20, weight=ft.FontWeight.BOLD),
    toolbar_text_style=ft.TextStyle(size=14),
    system_overlay_style=ft.SystemOverlayStyle.LIGHT,
),
navigation_bar_theme=ft.NavigationBarTheme(
    indicator_color=ft.Colors.BLUE,
    indicator_shape=ft.RoundedRectangleBorder(radius=10),
    label_behavior=ft.NavigationBarLabelBehavior.ALWAYS_SHOW,
    elevation=4,
    bgcolor=ft.Colors.WHITE,
    shadow_color=ft.Colors.GREY,
    height=56,
    label_text_style=ft.TextStyle(size=12),
    icon_size=24,
),
floating_action_button_theme=ft.FloatingActionButtonTheme(
    bgcolor=ft.Colors.BLUE,
    foreground_color=ft.Colors.WHITE,
    elevation=6,
    highlight_elevation=12,
    disabled_elevation=0,
    shape=ft.CircleBorder(),
    icon_size=24,
),
checkbox_theme=ft.CheckboxTheme(
    check_color=ft.Colors.WHITE,
    active_color=ft.Colors.BLUE,
    inactive_color=ft.Colors.GREY,
    hover_color=ft.Colors.BLUE_50,
    focus_color=ft.Colors.BLUE_100,
    splash_radius=20,
    shape=ft.RoundedRectangleBorder(radius=2),
    side=ft.BorderSide(color=ft.Colors.GREY, width=1),
```

```
    ),
    radio_theme=ft.RadioTheme(
        active_color=ft.Colors.BLUE,
        inactive_color=ft.Colors.GREY,
        hover_color=ft.Colors.BLUE_50,
        focus_color=ft.Colors.BLUE_100,
        splash_radius=20,
    ),
    switch_theme=ft.SwitchTheme(
        active_color=ft.Colors.BLUE,
        active_track_color=ft.Colors.BLUE_100,
        inactive_color=ft.Colors.GREY,
        inactive_track_color=ft.Colors.GREY_100,
        hover_color=ft.Colors.BLUE_50,
        focus_color=ft.Colors.BLUE_100,
        splash_radius=20,
    ),
    slider_theme=ft.SliderTheme(
        thumb_color=ft.Colors.BLUE,
        active_color=ft.Colors.BLUE,
        inactive_color=ft.Colors.GREY_300,
        overlay_color=ft.Colors.BLUE_50,
        hover_color=ft.Colors.BLUE_100,
        focus_color=ft.Colors.BLUE_200,
        track_height=2,
        thumb_radius=10,
        overlay_radius=20,
    ),
    text_field_theme=ft.TextFieldTheme(
        border=ft.InputBorder.OUTLINE,
        border_color=ft.Colors.GREY,
        focused_border_color=ft.Colors.BLUE,
        focused_border_width=2,
        cursor_color=ft.Colors.BLUE,
        selection_color=ft.Colors.BLUE,
        fill_color=ft.Colors.WHITE,
        hover_color=ft.Colors.GREY_50,
        bgcolor=ft.Colors.WHITE,
    ),
    chip_theme=ft.ChipTheme(
        bgcolor=ft.Colors.WHITE,
        selected_bgcolor=ft.Colors.BLUE,
        color=ft.Colors.BLUE,
        selected_color=ft.Colors.WHITE,
        shape=ft.RoundedRectangleBorder(radius=10),
        elevation=2,
        padding=ft.Padding(10, 5, 10, 5),
        label_padding=ft.Padding(5, 0, 5, 0),
    ),
    dialog_theme=ft.DialogTheme(
        bgcolor=ft.Colors.WHITE,
        elevation=24,
        shape=ft.RoundedRectangleBorder(radius=10),
        title_padding=ft.Padding(20, 20, 20, 0),
        content_padding=ft.Padding(20, 10, 20, 10),
        actions_padding=ft.Padding(20, 10, 20, 20),
    ),
    bottom_sheet_theme=ft.BottomSheetTheme(
        bgcolor=ft.Colors.WHITE,
        elevation=8,
        shape=ft.RoundedRectangleBorder(
            radius=ft.BorderRadius.only(top_left=10, top_right=10)
        ),
        show_drag_handle=True,
        drag_handle_color=ft.Colors.GREY,
        enable_drag=True,
    ),
)

# Set theme
page.theme = theme
page.dark_theme = dark_theme
page.theme_mode = ft.ThemeMode.SYSTEM
```

```

### ThemeMode Options

```python
page.theme_mode = ft.ThemeMode.LIGHT   # Force light theme
page.theme_mode = ft.ThemeMode.DARK    # Force dark theme
page.theme_mode = ft.ThemeMode.SYSTEM  # Follow system
```

ColorScheme

```python
color_scheme = ft.ColorScheme(
    primary=ft.Colors.BLUE,
    on_primary=ft.Colors.WHITE,
    primary_container=ft.Colors.BLUE_50,
    on_primary_container=ft.Colors.BLUE_900,
    secondary=ft.Colors.GREEN,
    on_secondary=ft.Colors.WHITE,
    secondary_container=ft.Colors.GREEN_50,
    on_secondary_container=ft.Colors.GREEN_900,
    tertiary=ft.Colors.ORANGE,
    on_tertiary=ft.Colors.WHITE,
    tertiary_container=ft.Colors.ORANGE_50,
    on_tertiary_container=ft.Colors.ORANGE_900,
    error=ft.Colors.RED,
    on_error=ft.Colors.WHITE,
    error_container=ft.Colors.RED_50,
    on_error_container=ft.Colors.RED_900,
    surface=ft.Colors.WHITE,
    on_surface=ft.Colors.BLACK,
    surface_variant=ft.Colors.GREY_50,
    on_surface_variant=ft.Colors.GREY_900,
    background=ft.Colors.GREY_50,
    on_background=ft.Colors.BLACK,
    outline=ft.Colors.GREY_400,
    outline_variant=ft.Colors.GREY_200,
    shadow=ft.Colors.BLACK,
    scrim=ft.Colors.BLACK,
    inverse_surface=ft.Colors.BLACK,
    on_inverse_surface=ft.Colors.WHITE,
    inverse_primary=ft.Colors.BLUE_200,
    surface_tint=ft.Colors.BLUE,
)
```

---

9. Window Management

Window Properties

```python
# Size
page.window.width = 800
page.window.height = 600
page.window.min_width = 400
page.window.min_height = 300
page.window.max_width = 1200
page.window.max_height = 900

# Behavior
page.window.resizable = True
page.window.full_screen = False
page.window.always_on_top = False
page.window.skip_task_bar = False
page.window.prevent_close = False
page.window.opacity = 1.0
page.window.frameless = False
page.window.draggable = True
page.window.toolbar_visible = True
page.window.title_bar_buttons_visible = True
```
```

```
# Title & Icon
page.window.title = "My App"
page.window.icon = "icon.ico"  # Windows
page.window.icon = "icon.icns"  # macOS

# Position
page.window.left = 100
page.window.top = 100

# Focus
page.window.focused = True

# Web window
page.web_window_name = "_blank"
```

Window Events

```python
def main(page: ft.Page):
    def on_resize(e: ft.WindowResizeEvent):
        print(f"Size: {e.width}x{e.height}")
    page.on_resize = on_resize

    def on_close(e: ft.WindowCloseEvent):
        print("Window closing")
        # Can prevent close
        # e.prevent_close = True
    page.on_close = on_close

    def on_focus(e: ft.WindowFocusEvent):
        print(f"Focused: {e.data}")
    page.on_window_focus = on_focus

    def on_blur(e: ft.WindowBlurEvent):
        print("Window blurred")
    page.on_window_blur = on_blur

    def on_maximize(e: ft.WindowMaximizeEvent):
        print(f"Maximized: {e.data}")
    page.on_window_maximize = on_maximize

    def on_minimize(e: ft.WindowMinimizeEvent):
        print(f"Minimized: {e.data}")
    page.on_window_minimize = on_minimize

    def on_restore(e: ft.WindowRestoreEvent):
        print("Window restored")
    page.on_window_restore = on_restore

ft.run(main)
```

Window Methods

```python
# Set window size
page.window.width = 800
page.window.height = 600
page.update()

# Set window position
page.window.left = 100
page.window.top = 100
page.update()

# Toggle full screen
page.window.full_screen = not page.window.full_screen
page.update()

# Toggle always on top
page.window.always_on_top = not page.window.always_on_top
```

```
page.update()

# Close window
page.window.destroy()

# Center window
page.window.center()
page.update()
```

---

10. State Management Deep Dive

Observables - Complete Example

```python
from dataclasses import dataclass
import flet as ft

@ft.observable
@dataclass
class TodoItem:
    id: int
    text: str
    completed: bool = False
    editing: bool = False

@ft.observable
@dataclass
class TodoApp:
    items: list[TodoItem] = ft.ObservableList()
    filter: str = "all"  # all, active, completed
    new_todo: str = ""
    next_id: int = 0

    def add_todo(self, text: str):
        if text.strip():
            self.items.append(TodoItem(id=self.next_id, text=text.strip()))
            self.next_id += 1
            self.new_todo = ""

    def delete_todo(self, item_id: int):
        self.items[:] = [item for item in self.items if item.id != item_id]

    def toggle_todo(self, item_id: int):
        for item in self.items:
            if item.id == item_id:
                item.completed = not item.completed
                break

    def get_filtered_items(self):
        if self.filter == "active":
            return [item for item in self.items if not item.completed]
        elif self.filter == "completed":
            return [item for item in self.items if item.completed]
        return self.items

@ft.component
def TodoItemView(item: TodoItem, app: TodoApp) -> ft.Control:
    is_editing, set_is_editing = ft.use_state(False)
    edit_text, set_edit_text = ft.use_state(item.text)

    def start_edit():
        set_edit_text(item.text)
        set_is_editing(True)

    def save_edit():
        item.text = edit_text
        set_is_editing(False)

    def cancel_edit():
        set_is_editing(False)
```

```
    if is_editing:
        return ft.Row([
            ft.TextField(
                value=edit_text,
                on_change=lambda e: set_edit_text(e.control.value),
                on_submit=lambda e: save_edit(),
                autofocus=True,
                width=300,
            ),
            ft.IconButton(ft.Icons.SAVE, on_click=save_edit),
            ft.IconButton(ft.Icons.CLOSE, on_click=cancel_edit),
        ])

    return ft.Row([
        ft.Checkbox(
            value=item.completed,
            on_change=lambda e: app.toggle_todo(item.id),
        ),
        ft.Text(
            item.text,
            style=ft.TextStyle(
                decoration=ft.TextDecoration.LINE_THROUGH if item.completed else None,
            ),
        ),
        ft.IconButton(ft.Icons.EDIT, on_click=start_edit),
        ft.IconButton(ft.Icons.DELETE, on_click=lambda: app.delete_todo(item.id)),
    ])

@ft.component
def TodoAppView() -> ft.Control:
    app, _ = ft.use_state(TodoApp())

    def add_todo(e):
        app.add_todo(app.new_todo)

    items = app.get_filtered_items()

    return ft.Column([
        ft.Text("Todo App", size=32, weight=ft.FontWeight.BOLD),
        ft.Row([
            ft.TextField(
                value=app.new_todo,
                on_change=lambda e: setattr(app, "new_todo", e.control.value),
                on_submit=add_todo,
                label="Add todo",
                width=300,
            ),
            ft.IconButton(ft.Icons.ADD, on_click=add_todo),
        ]),
        ft.Row([
            ft.TextButton(
                "All",
                on_click=lambda: setattr(app, "filter", "all"),
                style=ft.ButtonStyle(
                    bgcolor=ft.Colors.BLUE_50 if app.filter == "all" else None,
                ),
            ),
            ft.TextButton(
                "Active",
                on_click=lambda: setattr(app, "filter", "active"),
                style=ft.ButtonStyle(
                    bgcolor=ft.Colors.BLUE_50 if app.filter == "active" else None,
                ),
            ),
            ft.TextButton(
                "Completed",
                on_click=lambda: setattr(app, "filter", "completed"),
                style=ft.ButtonStyle(
                    bgcolor=ft.Colors.BLUE_50 if app.filter == "completed" else None,
                ),
            ),
        ]),
```

```
        ft.Column([TodoItemView(item, app) for item in items]),
        ft.Text(f"{len(app.items)} items total"),
    ])
```

use_effect Hook

```python
@ft.component
def Counter():
    count, set_count = ft.use_state(0)

    # useEffect runs after render
    ft.use_effect(
        lambda: print(f"Count changed to {count}"),
        dependencies=[count],  # Only run when count changes
    )

    # useEffect with cleanup
    ft.use_effect(
        lambda: (
            print(f"Started timer for count {count}"),
            lambda: print(f"Cleaned up timer for count {count}")
        ),
        dependencies=[count],
    )

    return ft.Row([
        ft.Text(f"Count: {count}"),
        ft.Button("Increment", on_click=lambda: set_count(count + 1)),
    ])
```

use_memo Hook

```python
@ft.component
def ExpensiveComputation():
    count, _ = ft.use_state(0)
    value, set_value = ft.use_state(0)

    # Computed value with memoization
    expensive_result = ft.use_memo(
        lambda: sum(range(value * 10000)),  # Expensive computation
        dependencies=[value],
    )

    return ft.Column([
        ft.Text(f"Value: {value}"),
        ft.Text(f"Expensive result: {expensive_result}"),
        ft.Button("Update value", on_click=lambda: set_value(value + 1)),
    ])
```

use_context and Provider

```python
# Define context
ThemeContext = ft.create_context({
    "theme": ft.ThemeMode.LIGHT,
    "toggle_theme": None,
})

@ft.component
def ThemedApp():
    theme, set_theme = ft.use_state(ft.ThemeMode.LIGHT)

    def toggle_theme():
        set_theme(ft.ThemeMode.DARK if theme == ft.ThemeMode.LIGHT else ft.ThemeMode.LIGHT)

    # Provide context to children
    return ft.Provider(
        value={"theme": theme, "toggle_theme": toggle_theme},
```

```
        content=ft.Column([
            ft.Text("App"),
            ThemedComponent(),
            ft.Button("Toggle theme", on_click=toggle_theme),
        ]),
    )

@ft.component
def ThemedComponent():
    # Consume context
    context = ft.use_context(ThemeContext)
    theme = context["theme"]
    toggle = context["toggle_theme"]

    return ft.Column([
        ft.Text(f"Current theme: {theme}"),
        ft.Button("Change from child", on_click=toggle),
    ])
```

---

11. Database Integration

SQLite Integration

```python
import sqlite3
from pathlib import Path

class Database:
    def __init__(self):
        self.db_path = Path("app.db")
        self.init_db()

    def init_db(self):
        with sqlite3.connect(self.db_path) as conn:
            conn.execute("""
                CREATE TABLE IF NOT EXISTS users (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    name TEXT NOT NULL,
                    email TEXT UNIQUE NOT NULL,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                )
            """)
            conn.commit()

    def get_user(self, user_id: int):
        with sqlite3.connect(self.db_path) as conn:
            conn.row_factory = sqlite3.Row
            result = conn.execute(
                "SELECT * FROM users WHERE id = ?",
                (user_id,)
            ).fetchone()
            return dict(result) if result else None

    def get_users(self):
        with sqlite3.connect(self.db_path) as conn:
            conn.row_factory = sqlite3.Row
            results = conn.execute("SELECT * FROM users").fetchall()
            return [dict(row) for row in results]

    def create_user(self, name: str, email: str):
        with sqlite3.connect(self.db_path) as conn:
            cursor = conn.execute(
                "INSERT INTO users (name, email) VALUES (?, ?)",
                (name, email)
            )
            conn.commit()
            return cursor.lastrowid

    def update_user(self, user_id: int, name: str, email: str):
        with sqlite3.connect(self.db_path) as conn:
```

```
            conn.execute(
                "UPDATE users SET name = ?, email = ? WHERE id = ?",
                (name, email, user_id)
            )
            conn.commit()

    def delete_user(self, user_id: int):
        with sqlite3.connect(self.db_path) as conn:
            conn.execute("DELETE FROM users WHERE id = ?", (user_id,))
            conn.commit()

db = Database()

@ft.component
def UserView(user: dict, on_delete) -> ft.Control:
    return ft.Row([
        ft.Text(f"{user['name']} ({user['email']})"),
        ft.IconButton(ft.Icons.DELETE, on_click=lambda: on_delete(user['id'])),
    ])

@ft.component
def AppView() -> ft.Control:
    users, set_users = ft.use_state(db.get_users())
    name, set_name = ft.use_state("")
    email, set_email = ft.use_state("")

    def add_user():
        db.create_user(name, email)
        set_users(db.get_users())
        set_name("")
        set_email("")

    def delete_user(user_id: int):
        db.delete_user(user_id)
        set_users(db.get_users())

    return ft.Column([
        ft.Text("Users", size=32),
        ft.Row([
            ft.TextField(
                value=name,
                on_change=lambda e: set_name(e.control.value),
                label="Name",
                width=200,
            ),
            ft.TextField(
                value=email,
                on_change=lambda e: set_email(e.control.value),
                label="Email",
                width=200,
            ),
            ft.Button("Add", on_click=add_user),
        ]),
        ft.Column([UserView(user, delete_user) for user in users]),
    ])
```

SQLAlchemy Integration

```python
from sqlalchemy import create_engine, Column, Integer, String, DateTime
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session
from datetime import datetime

Base = declarative_base()

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
    email = Column(String(100), unique=True, nullable=False)
```

```
    created_at = Column(DateTime, default=datetime.now)

engine = create_engine("sqlite:///app.db")
Base.metadata.create_all(engine)

SessionLocal = sessionmaker(bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@ft.component
def UserList(db: Session) -> ft.Control:
    users = db.query(User).all()
    # Render users...
```

MongoDB Integration

```python
from motor.motor_asyncio import AsyncIOMotorClient

class MongoDatabase:
    def __init__(self):
        self.client = AsyncIOMotorClient("mongodb://localhost:27017")
        self.db = self.client["flet_app"]
        self.users = self.db["users"]

    async def get_users(self):
        return await self.users.find().to_list(100)

    async def create_user(self, name: str, email: str):
        result = await self.users.insert_one({
            "name": name,
            "email": email,
            "created_at": datetime.now()
        })
        return result.inserted_id

    async def delete_user(self, user_id: str):
        from bson import ObjectId
        await self.users.delete_one({"_id": ObjectId(user_id)})

db = MongoDatabase()

@ft.component
async def MongoApp() -> ft.Control:
    users, set_users = ft.use_state([])
    name, set_name = ft.use_state("")
    email, set_email = ft.use_state("")

    async def load_users():
        set_users(await db.get_users())

    async def add_user():
        await db.create_user(name, email)
        await load_users()
        set_name("")
        set_email("")

    await load_users()

    return ft.Column([
        ft.Text("MongoDB Users", size=32),
        ft.Row([
            ft.TextField(
                value=name,
                on_change=lambda e: set_name(e.control.value),
                label="Name",
                width=200,
```

```
            ),
            ft.TextField(
                value=email,
                on_change=lambda e: set_email(e.control.value),
                label="Email",
                width=200,
            ),
            ft.Button("Add", on_click=add_user),
        ]),
        ft.Column([ft.Text(f"{user['name']} ({user['email']})") for user in users]),
    ])
```

---

12. Testing Flet Apps

Unit Tests with pytest

```python
# test_app.py
import pytest
import flet as ft
from flet.test import TestClient

class TestApp:
    def test_counter_app(self):
        def main(page: ft.Page):
            counter = ft.Text("0")

            def increment(e):
                counter.value = str(int(counter.value) + 1)
                page.update()

            page.add(
                counter,
                ft.Button("Increment", on_click=increment)
            )

        client = TestClient(main)

        # Test initial state
        assert client.page.controls[0].value == "0"

        # Click button
        client.page.controls[1].on_click(None)

        # Test updated state
        assert client.page.controls[0].value == "1"

    def test_to_do_app(self):
        @ft.component
        def TodoApp():
            todos, set_todos = ft.use_state([])
            new_todo, set_new_todo = ft.use_state("")

            def add_todo():
                if new_todo.strip():
                    set_todos(todos + [new_todo])
                    set_new_todo("")

            return ft.Column([
                ft.TextField(
                    value=new_todo,
                    on_change=lambda e: set_new_todo(e.control.value),
                    on_submit=add_todo,
                ),
                ft.Button("Add", on_click=add_todo),
                ft.Column([ft.Text(todo) for todo in todos]),
            ])

        client = TestClient(lambda page: page.render(TodoApp))
```

```
        # Add todo
        text_field = client.page.controls[0].controls[0]
        add_button = client.page.controls[0].controls[1]

        text_field.value = "Buy milk"
        text_field.on_change(None)  # Trigger on_change
        add_button.on_click(None)

        # Check todo was added
        todo_texts = client.page.controls[0].controls[2].controls
        assert len(todo_texts) == 1
        assert todo_texts[0].value == "Buy milk"

    def test_counter_with_custom_update(self):
        def main(page: ft.Page):
            counter = ft.Text("0")

            def increment(e):
                counter.value = str(int(counter.value) + 1)
                page.update()  # Manual update

            page.add(counter, ft.Button("Increment", on_click=increment))
            page.update()

        client = TestClient(main)

        # Click button multiple times
        for _ in range(5):
            client.page.controls[1].on_click(None)

        assert client.page.controls[0].value == "5"
```

Testing with Widgets

```python
def test_text_field():
    def main(page: ft.Page):
        text_field = ft.TextField(
            label="Name",
            value="John",
            on_change=lambda e: setattr(page, "changed_value", e.control.value),
        )
        page.add(text_field)
        page.update()

    client = TestClient(main)
    text_field = client.page.controls[0]

    # Test initial value
    assert text_field.value == "John"

    # Test change
    text_field.value = "Jane"
    text_field.on_change(None)
    assert client.page.changed_value == "Jane"

def test_dropdown():
    def main(page: ft.Page):
        dropdown = ft.Dropdown(
            options=[
                ft.dropdown.Option("option1", "Option 1"),
                ft.dropdown.Option("option2", "Option 2"),
            ],
            value="option1",
            on_change=lambda e: setattr(page, "selected", e.control.value),
        )
        page.add(dropdown)
        page.update()

    client = TestClient(main)
    dropdown = client.page.controls[0]
```

```
    # Test initial value
    assert dropdown.value == "option1"

    # Test change
    dropdown.value = "option2"
    dropdown.on_change(None)
    assert client.page.selected == "option2"
```

Testing with Mock Data

```python
class MockDatabase:
    def __init__(self):
        self.users = [
            {"id": 1, "name": "John", "email": "john@example.com"},
            {"id": 2, "name": "Jane", "email": "jane@example.com"},
        ]

    def get_users(self):
        return self.users

    def create_user(self, name, email):
        new_id = max([u["id"] for u in self.users]) + 1
        user = {"id": new_id, "name": name, "email": email}
        self.users.append(user)
        return new_id

    def delete_user(self, user_id):
        self.users = [u for u in self.users if u["id"] != user_id]

def test_user_app():
    db = MockDatabase()

    @ft.component
    def UserApp() -> ft.Control:
        users, set_users = ft.use_state(db.get_users())

        def delete_user(user_id):
            db.delete_user(user_id)
            set_users(db.get_users())

        return ft.Column([
            ft.Column([ft.Text(f"{u['name']}") for u in users]),
            *[ft.Button(f"Delete {u['name']}", on_click=lambda e, uid=u['id']: delete_user(uid)) for u in users],
        ])

    client = TestClient(lambda page: page.render(UserApp))

    # Test initial state
    user_texts = client.page.controls[0].controls[0].controls
    assert len(user_texts) == 2
    assert user_texts[0].value == "John"
    assert user_texts[1].value == "Jane"

    # Test delete
    delete_buttons = client.page.controls[0].controls[1:]
    delete_buttons[0].on_click(None)

    # Test updated state
    user_texts = client.page.controls[0].controls[0].controls
    assert len(user_texts) == 1
    assert user_texts[0].value == "Jane"
```

---

13. Environment Variables

Complete List of Flet Environment Variables

```python
import os
```

```
# Storage paths (set by Flet)
app_data = os.getenv("FLET_APP_STORAGE_DATA")
app_cache = os.getenv("FLET_APP_STORAGE_CACHE")
app_temp = os.getenv("FLET_APP_STORAGE_TEMP")
app_console = os.getenv("FLET_APP_CONSOLE")

# Web app settings
web_server = os.getenv("FLET_FORCE_WEB_SERVER", "false")
route_url_strategy = os.getenv("FLET_ROUTE_URL_STRATEGY", "path")

# WebSocket settings
ws_max_message_size = int(os.getenv("FLET_WS_MAX_MESSAGE_SIZE", "1048576"))

# Development settings
debug = os.getenv("FLET_DEBUG", "false")

# OAuth settings
oauth_callback_url = os.getenv("FLET_OAUTH_CALLBACK_URL", "/oauth_callback")

# Assets
assets_dir = os.getenv("FLET_ASSETS_DIR", "assets")

# Logging
log_level = os.getenv("FLET_LOG_LEVEL", "INFO")

# Python
python_path = os.getenv("PYTHONPATH", "")
```

Using Environment Variables in Your App

```python
import os
from pathlib import Path

class AppConfig:
    # Storage paths
    STORAGE_DATA = Path(os.getenv("FLET_APP_STORAGE_DATA", "./data"))
    STORAGE_CACHE = Path(os.getenv("FLET_APP_STORAGE_CACHE", "./cache"))
    STORAGE_TEMP = Path(os.getenv("FLET_APP_STORAGE_TEMP", "./temp"))
    STORAGE_CONSOLE = os.getenv("FLET_APP_CONSOLE", "console.log")

    # App settings
    DEBUG = os.getenv("FLET_DEBUG", "false").lower() == "true"
    WEB_SERVER = os.getenv("FLET_FORCE_WEB_SERVER", "false").lower() == "true"
    ROUTE_STRATEGY = os.getenv("FLET_ROUTE_URL_STRATEGY", "path")

    # API settings
    API_URL = os.getenv("API_URL", "https://api.example.com")
    API_KEY = os.getenv("API_KEY", "")

    # Database
    DB_PATH = STORAGE_DATA / "app.db"

    @classmethod
    def ensure_directories(cls):
        cls.STORAGE_DATA.mkdir(parents=True, exist_ok=True)
        cls.STORAGE_CACHE.mkdir(parents=True, exist_ok=True)
        cls.STORAGE_TEMP.mkdir(parents=True, exist_ok=True)

def main(page: ft.Page):
    # Initialize directories
    AppConfig.ensure_directories()

    # Use config
    page.title = "My App"

    if AppConfig.DEBUG:
        print("Running in debug mode")

    # Database path
    db_path = AppConfig.DB_PATH
```

```
    print(f"Database at: {db_path}")

    # API config
    api_url = AppConfig.API_URL
    api_key = AppConfig.API_KEY
```

Setting Environment Variables

```python
# Via command line (Linux/Mac)
# export FLET_APP_STORAGE_DATA="./my_app_data"
# export FLET_DEBUG="true"
# flet run main.py

# Via command line (Windows)
# set FLET_APP_STORAGE_DATA=./my_app_data
# set FLET_DEBUG=true
# flet run main.py

# Via Python (before app starts)
import os
os.environ["FLET_APP_STORAGE_DATA"] = "./my_app_data"
os.environ["FLET_DEBUG"] = "true"
```

---

14. Packaging All Platforms

Packaging for Android

```bash
# Build APK
flet build apk

# Build AAB (Google Play)
flet build aab

# Build with specific architectures
flet build apk --arch arm64-v8a x86_64

# Split APK per architecture
flet build apk --split-per-abi

# Sign the app
flet build aab \
    --android-signing-key-store ~/upload-keystore.jks \
    --android-signing-key-alias upload \
    --android-signing-key-store-password <password> \
    --android-signing-key-password <password>
```

Packaging for iOS

```bash
# Build IPA (requires macOS)
flet build ipa

# Build for iOS Simulator
flet build ios-simulator

# Sign the app (Info.plist configuration)
# Configure in pyproject.toml:
# [tool.flet.ios]
# bundle_identifier = "com.mycompany.myapp"
# version = "1.0.0"
# build_number = "1"
```

Packaging for Windows

```bash
```

```
# Build Windows executable
flet build windows

# Build with specific options
flet build windows --arch x64

# Exclude files
flet build windows --exclude .git .venv
```

Packaging for macOS

```bash
# Build macOS app
flet build macos

# Build with specific architectures
flet build macos --arch arm64 x86_64

# Universal binary
flet build macos --arch arm64 x86_64
```

Packaging for Linux

```bash
# Build Linux app
flet build linux

# Build with specific options
flet build linux --arch x86_64
```

Packaging for Web

```bash
# Build web app (static)
flet build web

# Build with specific options
flet build web --base-url /myapp
```

pyproject.toml Configuration

```toml
[tool.flet]
# General settings
target_arch = ["x86_64", "arm64"]  # For Android/macOS
permissions = ["location", "camera"]

# App settings
app_name = "My Flet App"
version = "1.0.0"
description = "My Flet application"

# Platform-specific settings
[tool.flet.android]
min_sdk_version = 24
target_sdk_version = 35
split_per_abi = true
adaptive_icon_background = "#0B6BFF"

[tool.flet.android.signing]
key_store = "~/upload-keystore.jks"
key_alias = "upload"

[tool.flet.ios]
bundle_identifier = "com.mycompany.myapp"
version = "1.0.0"
build_number = "1"

[tool.flet.windows]
```

```
app_icon = "icon.ico"
file_version = "1.0.0.0"

[tool.flet.macos]
bundle_identifier = "com.mycompany.myapp"
bundle_version = "1.0.0"
build_number = "1"

[tool.flet.linux]
app_icon = "icon.png"

[tool.flet.web]
base_url = "/myapp"

# Compilation settings
[tool.flet.compile]
app = true
packages = true

[tool.flet.cleanup]
app = true
packages = true
app_files = ["**/*.c", "**/*.h"]
package_files = ["**/*.pyi"]

# Flutter dependencies
[tool.flet.flutter.pubspec.dependencies]
# Add Flutter packages here
# pkg_name = "^1.2.3"

[tool.flet.flutter.pubspec.dependency_overrides]
# Override Flutter packages
# pkg_name = { path = "../pkg_name" }
```

CI/CD with GitHub Actions

```yaml
name: Build Flet App

on:
  push:
  pull_request:
  workflow_dispatch:

env:
  UV_PYTHON: 3.12
  PYTHONUTF8: 1
  FLET_CLI_NO_RICH_OUTPUT: 1

jobs:
  build:
    name: Build ${{ matrix.name }}
    runs-on: ${{ matrix.runner }}
    strategy:
      fail-fast: false
      matrix:
        include:
          - name: linux
            runner: ubuntu-latest
            build_cmd: "flet build linux"
            artifact_path: build/linux
            needs_linux_deps: true

          - name: macos
            runner: macos-26
            build_cmd: "flet build macos"
            artifact_path: build/macos
            needs_linux_deps: false

          - name: windows
            runner: windows-latest
            build_cmd: "flet build windows"
```

```
            artifact_path: build/windows
            needs_linux_deps: false

          - name: aab
            runner: ubuntu-latest
            build_cmd: "flet build aab"
            artifact_path: build/aab
            needs_linux_deps: false

          - name: apk
            runner: ubuntu-latest
            build_cmd: "flet build apk"
            artifact_path: build/apk
            needs_linux_deps: false

          - name: ipa
            runner: macos-26
            build_cmd: "flet build ipa"
            artifact_path: build/ipa
            needs_linux_deps: false

          - name: ios-simulator
            runner: macos-26
            build_cmd: "flet build ios-simulator"
            artifact_path: build/ios-simulator
            needs_linux_deps: false

          - name: web
            runner: ubuntu-latest
            build_cmd: "flet build web"
            artifact_path: build/web
            needs_linux_deps: false

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup uv
        uses: astral-sh/setup-uv@v6

      - name: Install Linux dependencies
        if: matrix.needs_linux_deps
        shell: bash
        run: |
          sudo apt update --allow-releaseinfo-change
          LINUX_DEPS="$(uv run flet --version --json | jq -r '.linux_dependencies | join(" ")')"
          sudo apt-get install -y --no-install-recommends $LINUX_DEPS
          sudo apt-get clean

      - name: Build app
        shell: bash
        run: |
          uv run ${{ matrix.build_cmd }} --yes --verbose

      - name: Upload Artifact
        uses: actions/upload-artifact@v5.0.0
        with:
          name: ${{ matrix.name }}-build-artifact
          path: ${{ matrix.artifact_path }}
          if-no-files-found: error
          overwrite: false
```

---

15. Icons Reference

Popular Material Icons

```python
# Navigation & Actions
ft.Icons.HOME
ft.Icons.HOME_FILLED
```

ft.Icons.ARROW_BACK
ft.Icons.ARROW_FORWARD
ft.Icons.ARROW_DROP_DOWN
ft.Icons.ARROW_DROP_UP
ft.Icons.ARROW_DROP_DOWN_CIRCLE
ft.Icons.ARROW_DROP_UP_CIRCLE
ft.Icons.MENU
ft.Icons.MORE_VERT
ft.Icons.MORE_HORIZ
ft.Icons.SETTINGS
ft.Icons.SETTINGS_OUTLINED
ft.Icons.INFO
ft.Icons.INFO_OUTLINED
ft.Icons.HELP
ft.Icons.HELP_OUTLINED
ft.Icons.CLOSE
ft.Icons.CLEAR
ft.Icons.ADD
ft.Icons.REMOVE
ft.Icons.DELETE
ft.Icons.DELETE_OUTLINED
ft.Icons.EDIT
ft.Icons.CREATE
ft.Icons.SAVE
ft.Icons.UNDO
ft.Icons.REDO

Content & Media
ft.Icons.SEARCH
ft.Icons.FILTER_LIST
ft.Icons.FILTER_LIST_ALT
ft.Icons.SORT
ft.Icons.SHARE
ft.Icons.SHARE_OUTLINED
ft.Icons.LINK
ft.Icons.COPY
ft.Icons.CUT
ft.Icons.PASTE
ft.Icons.PRINT
ft.Icons.DOWNLOAD
ft.Icons.UPLOAD
ft.Icons.ATTACH_FILE
ft.Icons.ATTACH_MONEY

Communication
ft.Icons.EMAIL
ft.Icons.EMAIL_OUTLINED
ft.Icons.PHONE
ft.Icons.PHONE_OUTLINED
ft.Icons.CHAT
ft.Icons.CHAT_OUTLINED
ft.Icons.MESSAGE
ft.Icons.MESSAGE_OUTLINED
ft.Icons.COMMENT
ft.Icons.COMMENT_OUTLINED
ft.Icons.SEND
ft.Icons.NOTIFICATIONS
ft.Icons.NOTIFICATIONS_OUTLINED
ft.Icons.NOTIFICATIONS_ACTIVE
ft.Icons.NOTIFICATIONS_OFF

People & Account
ft.Icons.PERSON
ft.Icons.PERSON_OUTLINED
ft.Icons.PERSON_FILLED
ft.Icons.ACCOUNT_CIRCLE
ft.Icons.ACCOUNT_CIRCLE_OUTLINED
ft.Icons.GROUP
ft.Icons.GROUP_OUTLINED
ft.Icons.GROUP_ADD
ft.Icons.FACE
ft.Icons.FACE_OUTLINED

Files & Folders
ft.Icons.FOLDER
ft.Icons.FOLDER_OUTLINED
ft.Icons.FOLDER_OPEN
ft.Icons.DESCRIPTION
ft.Icons.FILE_PRESENT
ft.Icons.ATTACHMENT
ft.Icons.FORMAT_LIST_BULLETED
ft.Icons.FORMAT_LIST_NUMBERED

Images & Media
ft.Icons.IMAGE
ft.Icons.IMAGE_OUTLINED
ft.Icons.PHOTO_CAMERA
ft.Icons.PHOTO_CAMERA_OUTLINED
ft.Icons.VIDEO_CAMERA
ft.Icons.VIDEO_CAMERA_OUTLINED
ft.Icons.MUSIC_NOTE
ft.Icons.MUSIC_OFF
ft.Icons.MIC
ft.Icons.MIC_OFF
ft.Icons.VOLUME_UP
ft.Icons.VOLUME_DOWN
ft.Icons.VOLUME_MUTE
ft.Icons.PLAY_ARROW
ft.Icons.PAUSE
ft.Icons.STOP
ft.Icons.SKIP_NEXT
ft.Icons.SKIP_PREVIOUS
ft.Icons.FAST_FORWARD
ft.Icons.FAST_REWIND

Location & Map
ft.Icons.LOCATION_ON
ft.Icons.LOCATION_OFF
ft.Icons.LOCATION_SEARCHING
ft.Icons.MAP
ft.Icons.MAP_OUTLINED
ft.Icons.PLACE
ft.Icons.ROUTE
ft.Icons.DIRECTIONS
ft.Icons.DIRECTIONS_BIKE
ft.Icons.DIRECTIONS_BOAT
ft.Icons.DIRECTIONS_BUS
ft.Icons.DIRECTIONS_CAR
ft.Icons.DIRECTIONS_TRAIN
ft.Icons.DIRECTIONS_WALK
ft.Icons.MY_LOCATION

Business & Office
ft.Icons.WORK
ft.Icons.WORK_OUTLINED
ft.Icons.BUSINESS
ft.Icons.BUSINESS_OUTLINED
ft.Icons.STORE
ft.Icons.STORE_OUTLINED
ft.Icons.SHOPPING_CART
ft.Icons.SHOPPING_CART_OUTLINED
ft.Icons.CREDIT_CARD
ft.Icons.CREDIT_CARD_OUTLINED
ft.Icons.CALCULATOR
ft.Icons.CALCULATOR_OUTLINED

Education & Science
ft.Icons.SCHOOL
ft.Icons.SCHOOL_OUTLINED
ft.Icons.BOOK
ft.Icons.BOOK_OUTLINED
ft.Icons.BOOKMARK
ft.Icons.BOOKMARK_OUTLINED
ft.Icons.BOOKMARK_BORDER
ft.Icons.SCIENCE
ft.Icons.SCIENCE_OUTLINED

ft.Icons.BIOTECH
ft.Icons.BIOTECH_OUTLINED

Hardware & Devices
ft.Icons.COMPUTER
ft.Icons.PHONE_ANDROID
ft.Icons.PHONE_IPHONE
ft.Icons.LAPTOP
ft.Icons.LAPTOP_OUTLINED
ft.Icons.TABLET
ft.Icons.TABLET_OUTLINED
ft.Icons.WATCH
ft.Icons.WATCH_OUTLINED
ft.Icons.HEADPHONES
ft.Icons.HEADPHONES_OUTLINED
ft.Icons.GAMEPAD
ft.Icons.GAMEPAD_OUTLINED
ft.Icons.PRINTER
ft.Icons.PRINTER_OUTLINED

Social & Sharing
ft.Icons.FAVORITE
ft.Icons.FAVORITE_OUTLINED
ft.Icons.FAVORITE_BORDER
ft.Icons.STAR
ft.Icons.STAR_OUTLINED
ft.Icons.ST